Prüfungsleistung Security by Design STG-TINF24F

Entwickelt in einer 2er-Gruppe eine kleine, aber vollständige Webanwendung in Form eines Terminbuchungssystems für eine Arztpraxis bestehend aus Frontend und Backend. Welche Programmiersprachen ihr jeweils verwendet, bleibt euch überlassen. Der Schwerpunkt liegt auf **Security by Design** sowie auf einer nachvollziehbaren und reproduzierbaren Lieferkette bis zum Deployment auf ein lokales Kubernetes-Cluster.

Zu den zentralen Themen gehören unter anderem:

- Threat Modeling
- SBOM
- SAST
- CI/CD-Pipeline
- Code Signing
- Secure Coding
- Least Privilege

Abweichungen von den Anforderungen sind möglich, sofern sie schlüssig begründet und reflektiert werden.

Erklärung:

1. **Projektstart und Anforderungsdefinition**
   Gefordert wird ein Terminbuchungssystem für eine Arztpraxis. Überlegt euch zunächst genau, in welchen Programmiersprachen die einzelnen Komponenten umgesetzt werden. Klärt außerdem, wie die Architektur gestaltet ist und welche Rollen darin vorkommen. Im Anschluss definiert ihr mindestens fünf Security Requirements, die verpflichtend umgesetzt werden müssen. Ergänzend erstellt ihr ein Architekturdiagramm, das die vollständige Systemlandschaft abbildet – von den Nutzer:innen über alle Komponenten bis hin zur CI/CD-Pipeline.

2. **Threat Modeling**
   Auf Basis der Projektarchitektur erstellt ihr ein Systemmodell mit Data Flow Diagram (DFD) und klar definierten Vertrauensgrenzen. Anschließend führt ihr mithilfe von STRIDE für jede Komponente und für jeden Datenfluss, der eine Vertrauensgrenze überschreitet, eine Bedrohungsanalyse durch. Dafür nutzt ihr mindestens eine Tabelle, wie sie in der Vorlesung behandelt wurde. Von den sechs STRIDE-Kategorien müssen lediglich drei Bedrohungen implementiert bzw. teilweise implementiert werden. Für die übrigen drei dokumentiert ihr nachvollziehbar, weshalb sie nicht umgesetzt wurden.

Identifiziert außerdem euer persönliches „Worst Case“-Szenario bezogen auf den perfekten Angriff und erstellt dafür einen Attack Tree, der hinreichend genug ist, dass Gegenmaßnahmen definiert werden können. Angriffskosten und Schadenskosten müssen nicht berechnet werden.

3. **Implementierung des Projekts**
   Das Projekt wird in einem Repository versioniert. Wie ihr euren Code strukturiert und mit Branches arbeitet, bleibt euch überlassen. Sobald Änderungen am Main-Branch vorgenommen werden, wird eine Pipeline ausgelöst. Der Pipeline-Trigger umfasst folgende Schritte:
   a. Erstellung einer SBOM
   b. SAST, SCA und Secret Scan
   c. Build und Container-Build
   d. Image Signing
   e. Quality Gate mit folgenden Bedingungen:
      i. Abbruch bei gefundenen Secrets
      ii. Abbruch bei Findings mit einem CVSS-Score ≥ 7.0 (bzw. HIGH oder CRITICAL)
      iii. Abbruch, wenn Signaturen fehlen oder nicht gültig sind

   Im Anschluss wird das Container-Image über die Pipeline in Kubernetes deployed. Dies soll über eine Zwischeninstanz passieren (Container-Registry).

4. **Hinweise zum Projekt**
   Achtet bei der Umsetzung besonders auf sicheren Code. Dazu gehören unter anderem saubere Fehlerbehandlung ohne Informationslecks, Eingabevalidierung und der Verzicht auf hartcodierte Secrets im Repository. Sicherheitsmechanismen wie eine gültige Anmeldung müssen integriert werden. Die Umsetzung von MFA ist nicht Pflicht, wird aber positiv berücksichtigt. Der erste Login eines Users soll der Admin sein. Das Prinzip des Least Privilege in Kubernetes setzt ihr wie folgt um:
   a. Eigener Namespace
   b. Minimaler Service Account
   c. Security Kontext mit
      i. runAsNonRoot: true
      ii. readOnlyRootFilesystem: true
      iii. allowPrivilegeEscalation: false
      iv. capabilities.drop
   d. Network Policy, mindestens restriktiv für eingehenden Traffic

5. **Dokumentation**
   Zum Abschluss gebt ihr eine Dokumentation ab. Die Seitenzahl ist nicht relevant – wichtig ist, dass alle Inhalte klar erklärt und nachvollziehbar begründet sind. Die Dokumentation soll grundsätzlich Folgendes enthalten:
   a. **Ziel, Architektur-Schaubild, Tech-Stack**
   b. **Beschreibung aller Komponenten im Schaubild**
   c. **Beschreibung Threat Modeling mit Systemmodell, DFD, STRIDE, Attack-Tree**
   d. **Erklärung der Anwendung mit allen Sicherheitsfeatures**
   e. **Erklärung aller nicht-sicheren Parts mit Begründung**
   f. **Erklärung des Deployment-Prozesses mit Bestandteilen der CI/CD-Pipeline**
   g. **Lessons-Learned**

6. **Präsentation**
   In der Prüfungswoche hat jede Gruppe 15 Minuten Zeit für eine Präsentation. Sie dient dazu, einen Überblick über euer Projekt zu geben und euch die Möglichkeit zu geben, eure Ergebnisse noch einmal vorzustellen und zu demonstrieren. Die Präsentation findet ausschließlich mit mir statt und nicht vor dem gesamten Kurs.

7. **Abgabe**
   Abgabe wird per Moodle gemacht.
   a. Repository am besten privat mit mir als Maintainer:
      i. /frontend/
      ii. /backend/
      iii. /infrastructure/
         1. Manifeste/Helm (k8s/)
      iv. Verzeichnis mit Pipeline-Konfiguration
      v. /evidence/ (Beachte: Wenn mehrere Images, dann auch mehrere evidence-Verzeichnisse)
         1. Run-Logs der Pipeline (run/)
         2. SAST-Berichte aller Runs (sast/)
         3. SBOM-Berichte aller Runs (sbom/)
         4. SCA-Berichte aller Runs (sca/)
         5. Secret-Berichte aller Runs (secrets/)
         6. Signing/Verify-Berichte aller Runs (signing/)
      vi. VERSION.txt
      vii. README.md mit ausführlichem Setup-Guide
      viii. Makefile
   b. Zip-File mit folgenden Inhalten (team-<nr>.zip):
      i. Ordner: Image
         1. Finales Image als .tar (oder mehrere Images)

2. image-ref.txt: vollständige Digest-Referenz (optional)
3. Alle endgültigen Berichte der Images
4. digest.txt: SHA-256 Digest
5. Public Keys der Images
6. image.sig oder image.bundle

ii. Ordner: Dokumentation
iii. Repository als .zip
iv. Ordner: Threat Modelling:
    1. Systemmodell mit DFD und Vertrauensgrenzen
    2. STRIDE-Bedrohungsanalyse
    3. Attack-Tree

Hinweis Makefile:

Zweck eines Makefiles ist eine komplette Blaupause des Projekts und gilt der Repoduzierbarkeit. Hier soll folgendes gemacht werden:

1. „make up“
2. Cluster hochfahren, Namespace anlegen, Ingress ausrollen
3. Image laden
4. Manifeste anwenden
5. Warten bis Kubernetes hochgefahren und healthy ist
6. „Ready“ ausgeben

Bewertungsmatrix:

Security Requirements	5 Punkte
Architektur (-diagramm)	5 Punkte
Konsistenz Architektur/Umsetzung	5 Punkte
Umsetzung Anforderungen	5 Punkte

Summe Anforderungen und Architektur: 20 Punkte

Systemmodell mit DFD und Vertrauensgrenzen	5 Punkte
STRIDE-Bedrohungsanalyse mit Umsetzung	6 Punkte
Attack Tree	4 Punkte

Summe Threat Modeling: 15 Punkte

Secure Coding	5 Punkte
Berechtigungskonzept	5 Punkte
Secret-Hygiene	5 Punkte

Summe Secure Coding & App-Security: 15 Punkte (+10 Punkte MFA)

SBOM-Erzeugung	5 Punkte
SAST/SCA/Secret-Scan	5 Punkte
Image-Signing	5 Punkte
Quality Gate	5 Punkte

Summe CI/CD: 20 Punkte

Security Kontext	5 Punkte
Namespace + Service Account	5 Punkte
Network Policy	5 Punkte

Summe Kubernetes Security: 15 Punkte

Dokumentation	8 Punkte
Repository und Reproduzierbarkeit	7 Punkte

Summe Abgabe: 15 Punkte

Anforderungen und Architektur (20)	+
Threat Modeling (15)	+
Secure Coding & App-Security (15+10)	+
CI/CD (20)	+
Kubernetes Security (15)	+
Abgabe (15)	+
Summe	100(110)